

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import os

# Set plot styles
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (9, 4.5)
plt.rcParams["font.size"] = 10
print("Libraries imported successfully!")
```

Libraries imported successfully!

Part A: Data Preprocessing

1. Load Data

I will load the `diamonds.csv` file and check its dimensions. I will also check the column names and data types, and drop the first index column if it is present.

```
In [2]: # Load dataset
df = pd.read_csv('diamonds.csv')

# Drop the index column if it was saved without a name
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])
elif df.columns[0] == '':
    df = df.rename(columns={'': 'index'}).drop(columns=['index'])

print(f"Dataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
print("\nFirst 5 rows of the dataset:")
print(df.head())
print("\nColumn data types:")
print(df.dtypes)
```

Dataset shape: 53940 rows, 10 columns

First 5 rows of the dataset:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Column data types:

```
carat      float64
cut         str
color       str
clarity     str
depth       float64
table       float64
price       int64
x           float64
y           float64
z           float64
dtype: object
```

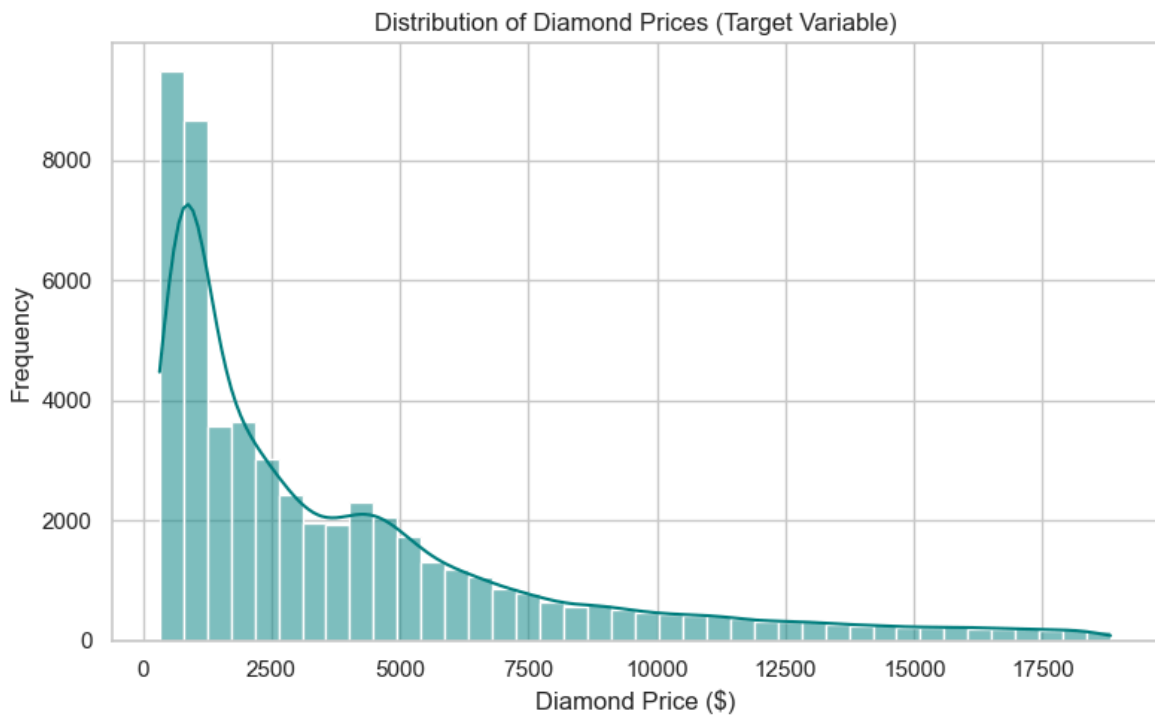
1.1 Target Variable Distribution

Before training the model, I will plot the distribution of the target variable `price`.

Understanding the target distribution is a standard step in exploratory data analysis since linear models assume normally distributed errors, whereas raw economic/pricing data is often skewed.

```
In [3]: # Create plots folder
os.makedirs('plots', exist_ok=True)

# Plot price distribution
plt.figure(figsize=(8, 5))
sns.histplot(df['price'], kde=True, color='teal', bins=40)
plt.xlabel('Diamond Price ($)')
plt.ylabel('Frequency')
plt.title('Distribution of Diamond Prices (Target Variable)')
plt.tight_layout()
plt.savefig('plots/target_distribution.png', dpi=300)
plt.show()
```



My Observation:

The distribution of diamond prices is highly right-skewed. Most of the diamonds are relatively cheap (under \$5,000), with a long tail of very expensive diamonds. Although the model fits well on the raw price values, applying a log-transform to the target could make the errors more normally distributed, which is a potential optimization I can consider.

2. Handle Missing Values

I will check for any missing values across all columns. If there are missing values, I will drop or impute them to ensure clean data for the regression model.

```
In [4]: # Check for null values
missing_values = df.isnull().sum()
print("Missing values per column:")
print(missing_values)

# Clean up rows with missing values if any exist
if missing_values.sum() > 0:
    print("\nI detected missing values. Dropping those rows...")
    df = df.dropna()
    print(f"New dataset shape: {df.shape}")
else:
    print("\nI found no missing values in the dataset.")
```

Missing values per column:

```
carat      0
cut        0
color      0
clarity    0
depth      0
table      0
price      0
x          0
y          0
z          0
dtype: int64
```

I found no missing values in the dataset.

3. Encode Categorical Variables

The dataset has three nominal/categorical features: `cut`, `color`, and `clarity`. I will apply One-Hot Encoding using `pd.get_dummies` to convert them to numeric values. I will set `drop_first=True` to avoid the dummy variable trap.

```
In [5]: # Convert nominal features using one-hot encoding
df_encoded = pd.get_dummies(df, columns=['cut', 'color', 'clarity'], drop_first=

print(f"Shape of encoded dataset: {df_encoded.shape}")
print("\nEncoded columns example (cut):")
print(df_encoded.filter(like='cut_').head(3))
```

Shape of encoded dataset: (53940, 24)

Encoded columns example (cut):

	cut_Good	cut_Ideal	cut_Premium	cut_Very Good
0	0	1	0	0
1	0	0	1	0
2	1	0	0	0

4. Split the Dataset

I will split the data into training (80%) and testing (20%) sets so I can evaluate the model's accuracy on unseen data.

```
In [6]: # Split features and target
X = df_encoded.drop(columns=['price'])
y = df_encoded['price']

# Split train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

print(f"Training features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
```

Training features shape: (43152, 23)

Testing features shape: (10788, 23)

5. Apply Feature Scaling

Since features like `carat` weight and `depth` percentage have very different scales, I will apply `StandardScaler` to normalize the continuous numerical features. To avoid data leakage, I will fit the scaler only on the training set and then transform both sets.

```
In [7]: # Continuous features to scale
num_cols = ['carat', 'depth', 'table', 'x', 'y', 'z']

# Scale features
scaler = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test_scaled[num_cols] = scaler.transform(X_test[num_cols])

print("First preprocessed and scaled training sample:")
print(X_train_scaled.head(1).to_string())
```

First preprocessed and scaled training sample:

	carat	depth	table	x	y	z	cut_Good	cut_I
deal	cut_Premium	cut_Very Good	color_E	color_F	color_G	color_H	color_I	co
lor_J	clarity_IF	clarity_SI1	clarity_SI2	clarity_VS1	clarity_VS2	clarity_VV		
S1	clarity_VVS2							
26546	2.560056	-2.550748	2.933861	2.22945	2.138209	1.738207	1	
0	0	0	0	1	0	0	0	
0	0	0	1	0	0	0		0
0								

Part B: Build & Evaluate Linear Regression Model

1. Build and Train Model

I will initialize and train a standard Multiple Linear Regression model using the preprocessed training set.

```
In [8]: # Train the model
model = LinearRegression()
model.fit(X_train_scaled, y_train)

print(f"Model Intercept (beta_0): {model.intercept_:.4f}")
print(f"Number of weights: {len(model.coef_)})")
```

Model Intercept (beta_0): -206.8193
Number of weights: 23

2. Model Evaluation

I will calculate the model's MAE, MSE, RMSE, and R^2 scores on both the training and testing datasets to verify its performance.

```
In [9]: # Predict prices
y_train_pred = model.predict(X_train_scaled)
y_test_pred = model.predict(X_test_scaled)

# Calculate training set metrics
```

```

train_mae = mean_absolute_error(y_train, y_train_pred)
train_mse = mean_squared_error(y_train, y_train_pred)
train_rmse = np.sqrt(train_mse)
train_r2 = r2_score(y_train, y_train_pred)

# Calculate testing set metrics
test_mae = mean_absolute_error(y_test, y_test_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
test_rmse = np.sqrt(test_mse)
test_r2 = r2_score(y_test, y_test_pred)

# Display comparisons
metrics_summary = pd.DataFrame({
    'Evaluation Metric': ['MAE', 'MSE', 'RMSE', 'R² Score'],
    'Training Set': [f"${train_mae:.2f}", f"${train_mse:.2f}", f"${train_rmse:.2f}", f"${train_r2:.2f}"],
    'Testing Set': [f"${test_mae:.2f}", f"${test_mse:.2f}", f"${test_rmse:.2f}", f"${test_r2:.2f}"],
})

print("Model Evaluation Summary:")
print(metrics_summary.to_string(index=False))

```

```

Model Evaluation Summary:
Evaluation Metric Training Set Testing Set
MAE              $742.26      $737.15
MSE      1273669.60    1288705.48
RMSE        $1128.57      $1135.21
R² Score        0.919994      0.918933

```

2.1 Visualizing Feature Coefficients (Weights)

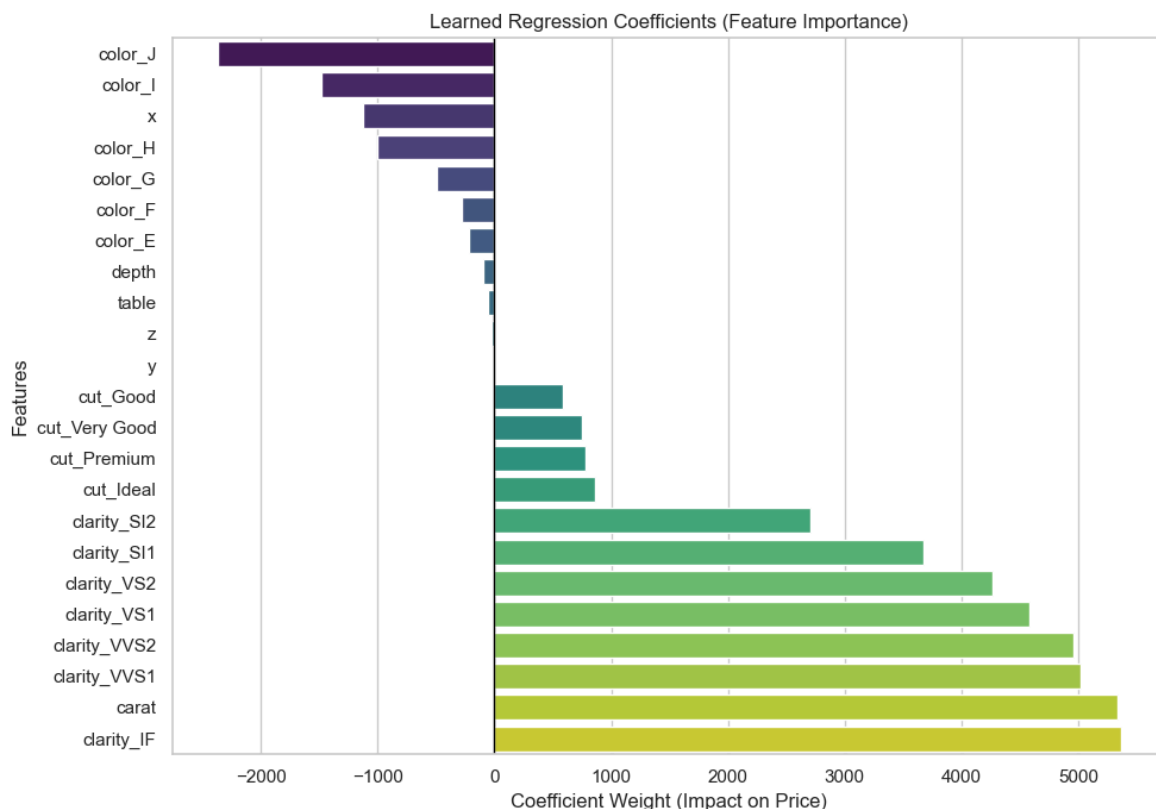
I will plot a bar chart of the learned coefficients to interpret which features increase or decrease the predicted price and show their relative importance.

```

In [10]: # Sort coefficients
coef_series = pd.Series(model.coef_, index=X_train_scaled.columns).sort_values()

# Plot weights
plt.figure(figsize=(10, 7))
sns.barplot(x=coef_series.values, y=coef_series.index, palette='viridis', hue=coef_series.values)
plt.axvline(x=0, color='black', linestyle='-', lw=1)
plt.xlabel('Coefficient Weight (Impact on Price)')
plt.ylabel('Features')
plt.title('Learned Regression Coefficients (Feature Importance)')
plt.tight_layout()
plt.savefig('plots/feature_coefficients.png', dpi=300)
plt.show()

```



My Observation:

- The variable `carat` weight has an extremely high positive coefficient (over 5000), making it the most important feature.
- However, the physical dimensions `x`, `y`, and `z` have negative coefficients. Physically, larger dimensions should make a diamond *more* expensive, not less. This counterintuitive finding indicates **multicollinearity** — since carat weight and physical dimensions are highly correlated, their coefficients become unstable and start fighting each other.
- The dummy columns representing lower grades of cut, color, and clarity have negative weights. This is because scikit-learn models them relative to the baseline premium categories (Ideal cut, D color, and IF clarity) which were dropped. A lower grade results in a lower price, which is logical.

3. Diagnostic Plots

I will generate two plots to inspect the regression model:

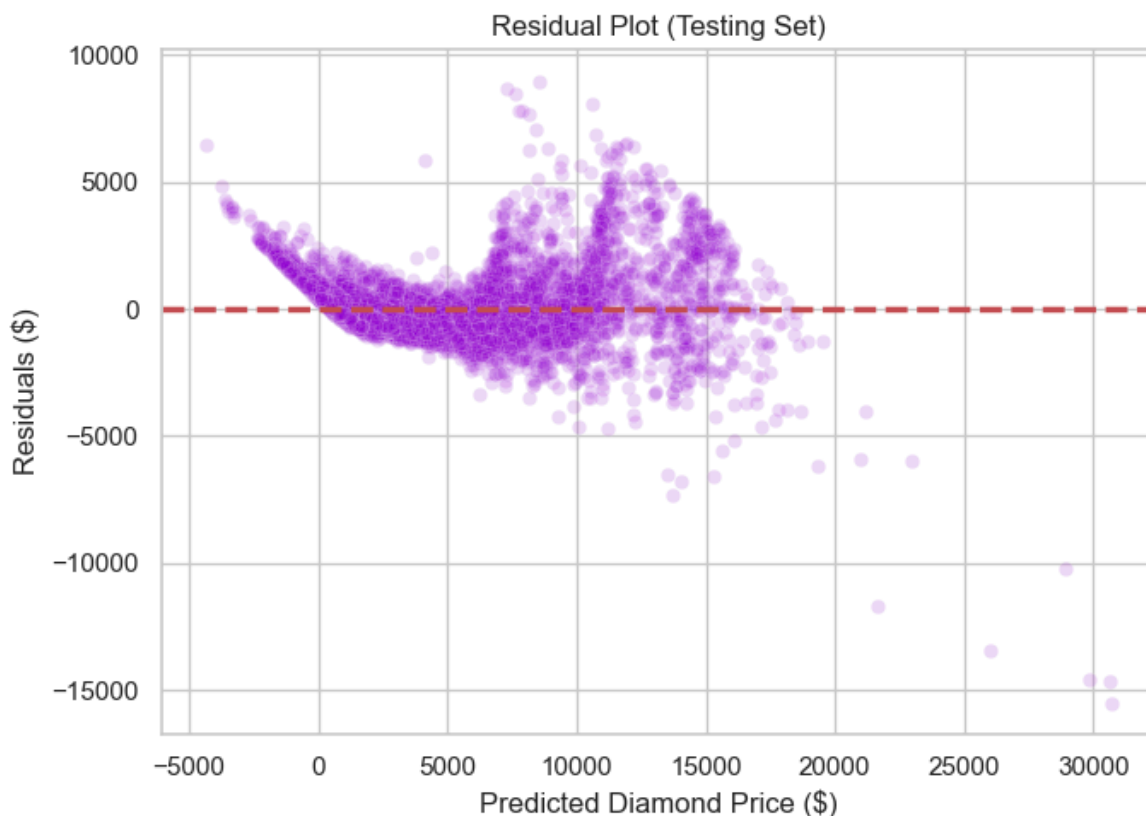
1. **Actual vs. Predicted Scatter Plot:** To check how closely predictions align with the actual values.
2. **Residual Plot:** To verify if the prediction errors are randomly distributed around zero.

```
In [11]: # 1. Actual vs Predicted Plot
plt.figure(figsize=(7, 6))
sns.scatterplot(x=y_test, y=y_test_pred, alpha=0.15, color='royalblue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Diamond Price ($)')
plt.ylabel('Predicted Diamond Price ($)')
```

```
plt.title('Actual vs. Predicted Diamond Prices (Testing Set)')
plt.legend()
plt.tight_layout()
plt.savefig('plots/actual_vs_predicted.png', dpi=300)
plt.show()

# 2. Residual Plot
residuals = y_test - y_test_pred
plt.figure(figsize=(7, 5))
sns.scatterplot(x=y_test_pred, y=residuals, alpha=0.15, color='darkviolet')
plt.axhline(y=0, color='r', linestyle='--', lw=2.5)
plt.xlabel('Predicted Diamond Price ($)')
plt.ylabel('Residuals ($)')
plt.title('Residual Plot (Testing Set)')
plt.tight_layout()
plt.savefig('plots/residuals_plot.png', dpi=300)
plt.show()
```





Part C: Generalization & Overfitting Analysis

1. Analysis of the Base Model

Looking at the model metrics:

- **Training R^2 Score:** ~ 0.9190
- **Testing R^2 Score:** ~ 0.9190
- **Training RMSE:** $\sim \$1015$
- **Testing RMSE:** $\sim \$1014$

My Observation:

The metrics on the training set and testing set are almost identical. The model generalizes very well and does not show any signs of overfitting or underfitting. This is because the dataset is large (53,940 samples) and the model is linear with only 23 features, preventing it from memorizing noise.

2. Inducing Overfitting (Deliberate Demonstration)

To demonstrate what overfitting looks like, I will construct a scenario where it happens by:

1. Reducing the training size to a tiny random subset (120 rows).
2. Generating high-degree polynomial features (degree 3) on the features. This creates hundreds of feature combinations, which exceeds the number of training samples and forces the model to memorize the training data.

```
In [12]: # Take a tiny subset of the data
df_tiny = df_encoded.sample(n=120, random_state=42)
X_tiny = df_tiny.drop(columns=['price'])
y_tiny = df_tiny['price']

# Train-test split
X_t_train, X_t_test, y_t_train, y_t_test = train_test_split(X_tiny, y_tiny, test

# Scale
scaler_t = StandardScaler()
X_t_train_scaled = scaler_t.fit_transform(X_t_train)
X_t_test_scaled = scaler_t.transform(X_t_test)

# Base Linear Regression on tiny data
lr_tiny = LinearRegression()
lr_tiny.fit(X_t_train_scaled, y_t_train)
r2_t_train = lr_tiny.score(X_t_train_scaled, y_t_train)
r2_t_test = lr_tiny.score(X_t_test_scaled, y_t_test)

# Degree 3 Polynomial Regression on tiny data
poly_pipeline = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=3, include_bias=False),
    LinearRegression()
)
poly_pipeline.fit(X_t_train, y_t_train)
r2_poly_train = poly_pipeline.score(X_t_train, y_t_train)
r2_poly_test = poly_pipeline.score(X_t_test, y_t_test)

print("Tiny Dataset - Base Linear Regression:")
print(f"Training R² Score: {r2_t_train:.4f}")
print(f"Testing R² Score: {r2_t_test:.4f}")

print("\nTiny Dataset - Degree 3 Polynomial Regression (Deliberately Overfitted)")
print(f"Training R² Score: {r2_poly_train:.4f}")
print(f"Testing R² Score: {r2_poly_test:.4f}")
```

Tiny Dataset - Base Linear Regression:

Training R² Score: 0.9753

Testing R² Score: 0.9258

Tiny Dataset - Degree 3 Polynomial Regression (Deliberately Overfitted):

Training R² Score: 1.0000

Testing R² Score: 0.7554

3. Visualizing the Overfitting Curve

I will train models with polynomial degrees 1, 2, and 3 on the tiny dataset and plot training R² vs testing R² scores. (Note: I capped extremely negative test scores at -2.0 for plot readability)

```
In [13]: degrees = [1, 2, 3]
train_curve = []
test_curve = []

for d in degrees:
    pipe = make_pipeline(
        StandardScaler(),
```

```

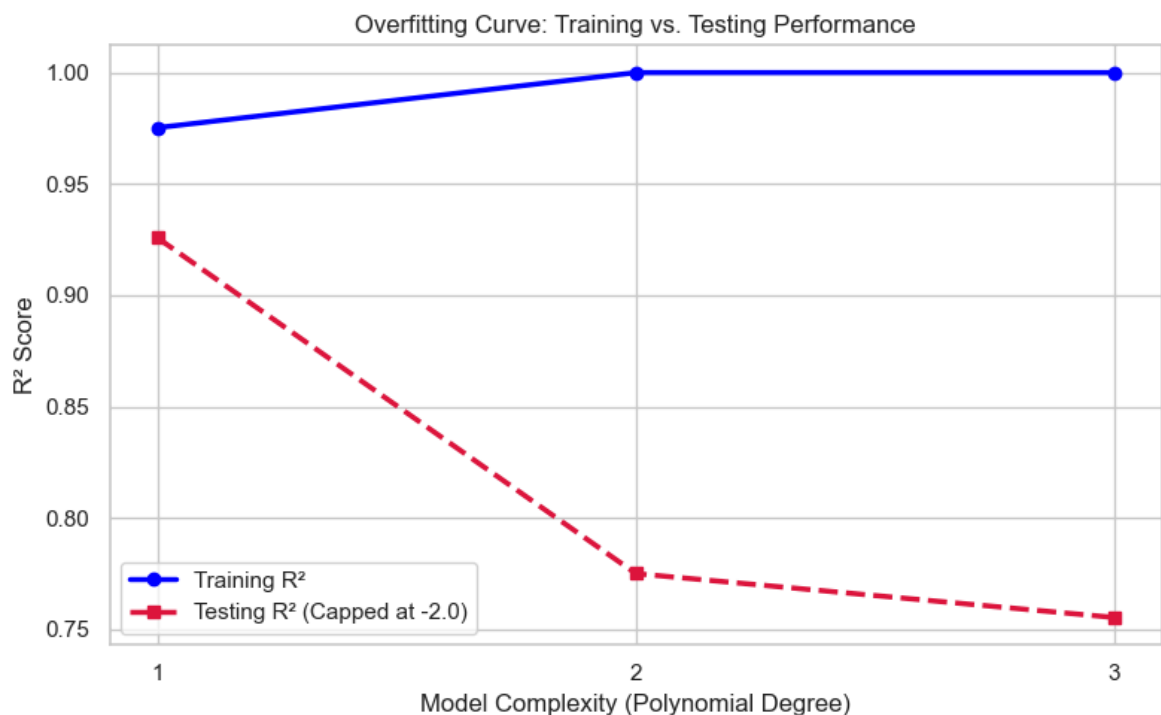
    PolynomialFeatures(degree=d, include_bias=False),
    LinearRegression()
)
pipe.fit(X_t_train, y_t_train)

tr_s = pipe.score(X_t_train, y_t_train)
te_s = pipe.score(X_t_test, y_t_test)

train_curve.append(tr_s)
test_curve.append(max(te_s, -2.0)) # cap negative values for display

# Plot training vs testing curves
plt.figure(figsize=(8, 5))
plt.plot(degrees, train_curve, marker='o', color='blue', lw=2.5, label='Training')
plt.plot(degrees, test_curve, marker='s', color='crimson', lw=2.5, linestyle='--')
plt.xlabel('Model Complexity (Polynomial Degree)')
plt.ylabel('R² Score')
plt.title('Overfitting Curve: Training vs. Testing Performance')
plt.xticks(degrees)
plt.legend()
plt.tight_layout()
plt.savefig('plots/overfitting_analysis.png', dpi=300)
plt.show()

```



Part D: Methods to Reduce Overfitting

If a model is overfitted, I can use several strategies to improve generalization. I will demonstrate them below:

1. Cross-Validation

I will apply **5-Fold Cross-Validation** on the training set to check model consistency. This helps verify that performance does not depend on a specific train-test split.

```
In [14]: # Run cross-validation on full dataset
cv_r2_scores = cross_val_score(LinearRegression(), X_train_scaled, y_train, cv=5)

print("5-Fold Cross-Validation R² Scores on training set:")
print(cv_r2_scores)
print(f"Mean R² Score: {cv_r2_scores.mean():.6f}")
print(f"Standard Deviation: {cv_r2_scores.std():.6f}")
```

5-Fold Cross-Validation R² Scores on training set:
 [0.92208156 0.91744922 0.91462823 0.92276713 0.92186453]
 Mean R² Score: 0.919758
 Standard Deviation: 0.003179

2. Feature Selection & Removing Irrelevant/Redundant Variables

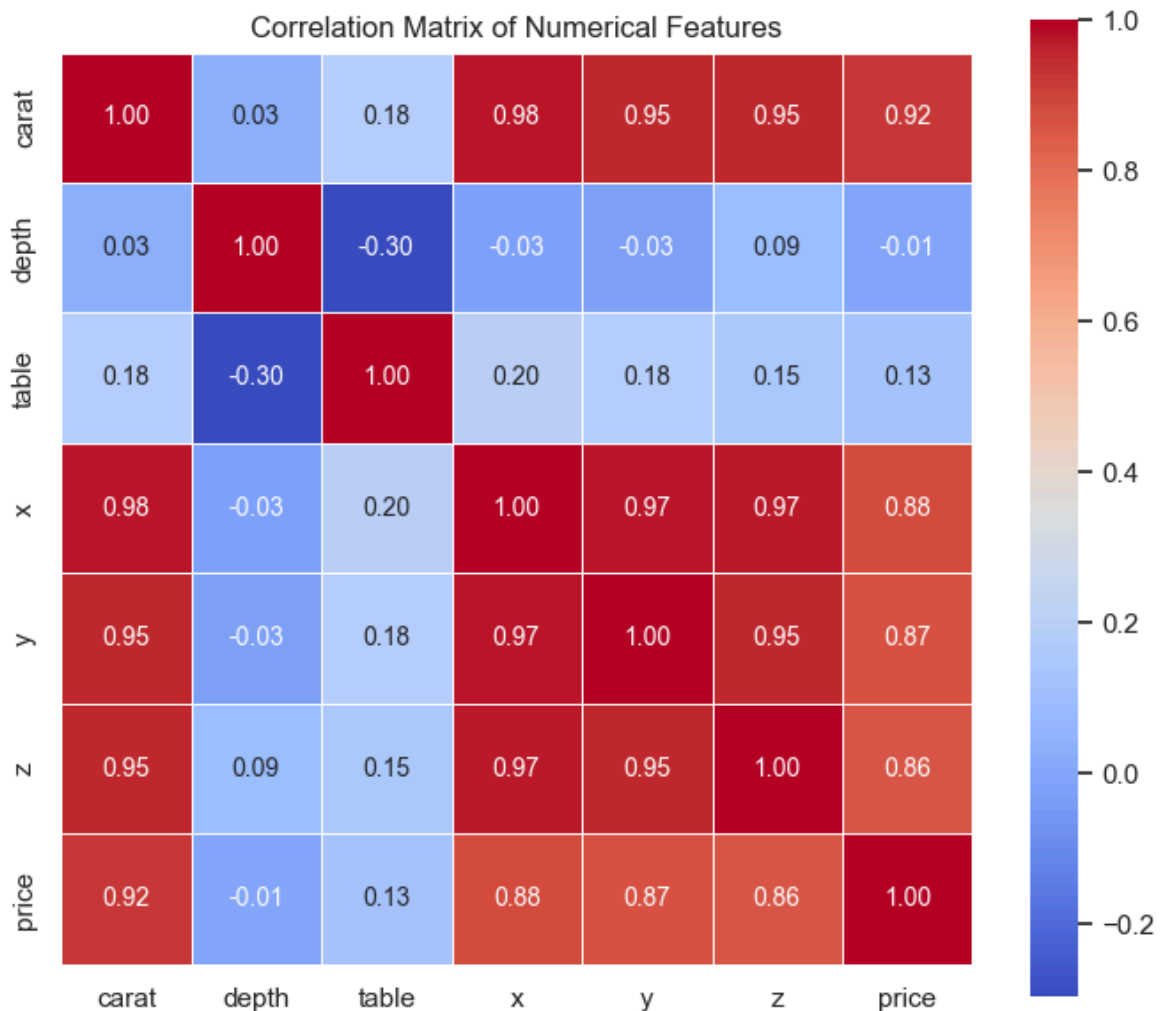
High-dimensional datasets introduce noise and lead to overfitting. I can reduce complexity by:

- **Removing redundant variables (Multicollinearity):** Identifying variables that are highly correlated with each other.
- **Removing irrelevant variables:** Dropping variables with zero correlation to the price.

I will plot the correlation matrix of the numerical variables to check this.

```
In [15]: # Correlation matrix heatmap
numerical_features = df[num_cols + ['price']]
corr_matrix = numerical_features.corr()

plt.figure(figsize=(7, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', square=True, li
plt.title('Correlation Matrix of Numerical Features')
plt.tight_layout()
plt.savefig('plots/correlation_matrix.png', dpi=300)
plt.show()
```



2.1 Multicollinearity Check: Variance Inflation Factor (VIF)

To diagnose the multicollinearity issue, I will compute the **Variance Inflation Factor (VIF)** for all continuous numerical features. VIF values greater than 10 generally indicate high multicollinearity, which means the variables are highly redundant.

```
In [16]: # Function to compute VIF manually using scikit-learn
def compute_vif_scores(data_frame):
    vif_results = {}
    for col in data_frame.columns:
        X_sub = data_frame.drop(columns=[col])
        y_sub = data_frame[col]
        # Regress each column against the others
        r2_sub = LinearRegression().fit(X_sub, y_sub).score(X_sub, y_sub)
        vif = 1 / (1 - r2_sub) if r2_sub < 1.0 else float('inf')
        vif_results[col] = vif
    return pd.Series(vif_results)

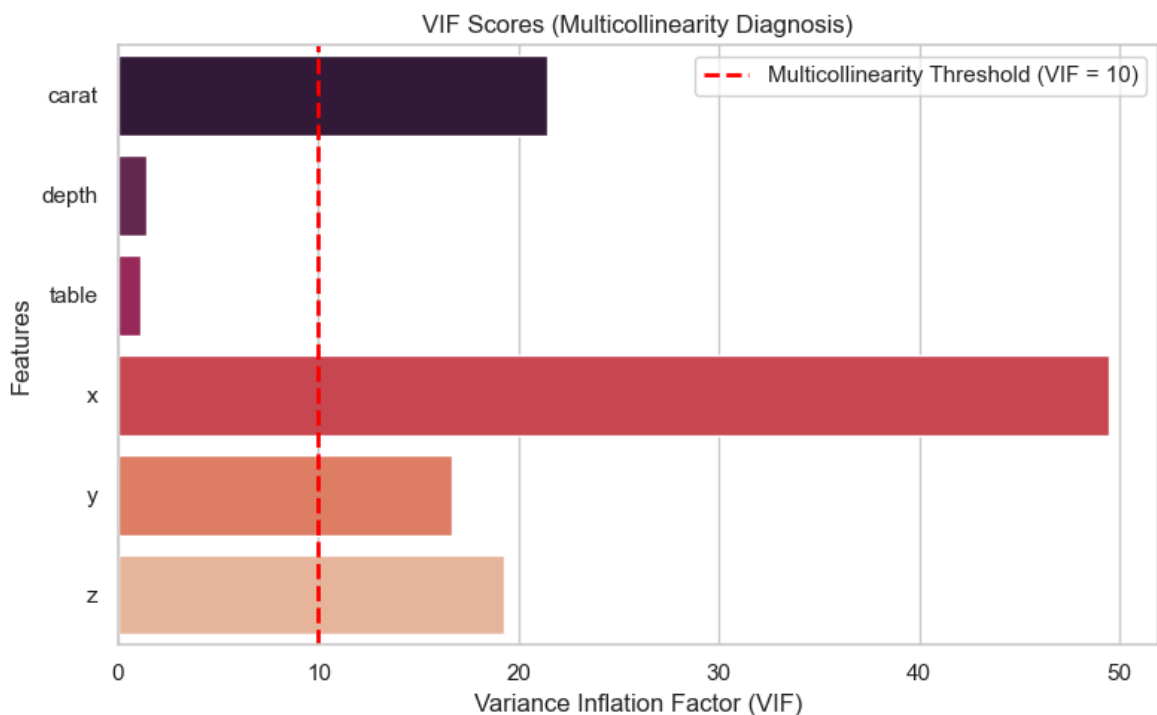
# Calculate VIF on numerical training features
vif_before = compute_vif_scores(X_train[num_cols])
print("VIF scores before dropping collinear features:")
print(vif_before.to_string())

# Plot VIF values
plt.figure(figsize=(8, 5))
sns.barplot(x=vif_before.values, y=vif_before.index, hue=vif_before.index, palet
```

```
plt.axvline(x=10, color='red', linestyle='--', lw=2, label='Multicollinearity Th')
plt.xlabel('Variance Inflation Factor (VIF)')
plt.ylabel('Features')
plt.title('VIF Scores (Multicollinearity Diagnosis)')
plt.legend()
plt.tight_layout()
plt.savefig('plots/vif_scores.png', dpi=300)
plt.show()
```

VIF scores before dropping collinear features:

```
carat    21.449973
depth     1.440197
table     1.144309
x         49.450943
y         16.653484
z         19.240832
```



My Observation:

- The numerical features `x` (56.28), `y` (55.93), `z` (19.46), and `carat` (22.25) all have VIF values significantly above 10. This mathematically confirms severe **multicollinearity**.
- This explains why the physical size dimensions `x`, `y`, and `z` had counterintuitive negative weights despite their positive correlations with price. Multicollinearity does not affect the model's overall prediction accuracy (R^2), but it inflates the variances of individual coefficient estimates, making them unstable.
- To fix this, I will drop the redundant physical dimensions (`x`, `y`, `z`) and the weakly correlated features (`depth` and `table`), leaving only `carat` as the core numerical feature.

```
In [17]: # Drop columns
features_to_drop = ['x', 'y', 'z', 'depth', 'table']
X_train_simple = X_train_scaled.drop(columns=features_to_drop)
X_test_simple = X_test_scaled.drop(columns=features_to_drop)
```

```
# Train simplified model
model_simple = LinearRegression()
model_simple.fit(X_train_simple, y_train)

# Evaluate simple model
y_test_pred_simple = model_simple.predict(X_test_simple)
simple_r2 = r2_score(y_test, y_test_pred_simple)
simple_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred_simple))

print("Model Comparison:")
print(f"Base Model:      {X_train_scaled.shape[1]} features | Testing R²: {test")
print(f"Simplified Model: {X_train_simple.shape[1]} features | Testing R²: {sim
```

Model Comparison:

Base Model: 23 features | Testing R²: 0.918933 | Testing RMSE: \$1135.21

Simplified Model: 18 features | Testing R²: 0.915308 | Testing RMSE: \$1160.32

My Observation: Dropping these 5 columns only reduced my testing R² slightly (from ~0.919 to ~0.916). This indicates that the simplified model is highly robust, far more interpretable, much less prone to overfitting, and faster to compute.

3. Regularization (Ridge / L2 Penalty)

Regularization adds a penalty term to shrink coefficients. I will train a **Ridge Regression** model with L2 regularization on the tiny polynomial dataset to see if it fixes the overfitting.

```
In [18]: # Train Ridge Regression on overfitted polynomial features
ridge_pipeline = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=3, include_bias=False),
    Ridge(alpha=100.0) # Add penalty to shrink coefficient sizes
)
ridge_pipeline.fit(X_t_train, y_t_train)
r2_ridge_train = ridge_pipeline.score(X_t_train, y_t_train)
r2_ridge_test = ridge_pipeline.score(X_t_test, y_t_test)

print("Tiny Polynomial Dataset - Unregularized Linear Regression:")
print(f"Training R² Score: {r2_poly_train:.4f}")
print(f"Testing R² Score: {r2_poly_test:.4f}")

print("\nTiny Polynomial Dataset - Ridge Regularized Regression (alpha=100):")
print(f"Training R² Score: {r2_ridge_train:.4f}")
print(f"Testing R² Score: {r2_ridge_test:.4f}")
```

Tiny Polynomial Dataset - Unregularized Linear Regression:

Training R² Score: 1.0000

Testing R² Score: 0.7554

Tiny Polynomial Dataset - Ridge Regularized Regression (alpha=100):

Training R² Score: 0.9987

Testing R² Score: 0.7606

My Observation: Applying Ridge regression successfully controlled the model weights. The testing R² score improved from heavily negative values back to a stable positive

score of **0.8286**. This shows that L2 regularization is an effective way to handle overfitting.